COGS 45 Lab 3 Assignment — RNNs.

In this lab, we train a **Recurrent Neural Network (RNN**) to iteratively predict the next character in a string of text, trained only using the works of William Shakespeare, as in Andre Karpathy's work.

****NOTE: training this model will take a few hours to train, so please don't wait until the absolute last minute to start this week's assignment!****.

The feedforward neural networks that we studied in the first two classes have two sources of information: the ***data*** that serves as the input, and the ***weights/biases*** that serve as the network's stored knowledge, acquired from previous experience. RNN's have these two sources, but are able to process **sequences of data**, allowing the early parts of the sequence to change the way that later parts of the sequence are represented. So representations are allowed to vary by context: e.g., different interpretations of "*time **flies like** an arrow"* vs. "*fruit **flies like** a banana."*

Here, we use an RNN to predict which **character** will come next in a string of text (i.e., the likelihood of each letter in the alphabet, given the characters that have been observed in the sequence thus far). In some cases, this is simple; for example, in English, U always follows Q with high probability. But it's unclear whether a system that begins with no knowledge and simply learns to predict the next character in a body of text can achieve human-like language competence (i.e., the syntax (the *structure* of language) or semantics (the *meaning* of language) necessary to jointly make new sentences comprehensible).

1. Train an RNN with the works of William Shakespeare as input, using the code provided (Week3_Lab_Code.ipynb) and the training data "Shakespeare.txt". *Expect this training to take a couple of hours*. Closing/Sleeping your computer can disrupt training. However, I have included code to **save** after every epoch. You can **re-load** the trained model by setting the "load_chk" parameter to "True". (Your model will likely be better the longer you train it!).

2. Generate and report samples periodically so we can get a *qualitative* sense of whether the network is improving. Choose some samples that you think are illustrative of the network's current abilities, at that training stage.

Sample Number: 6 Loss: 2.31863646
-Yools bastilg: and utastuet
were wourdt I blive citine not the wive mathler
he
the froocuiuting his?----------

Report: At this early stage, the network has learned basic characters and patterns, showing space and how a poem is structured (it has learned poems consist of a few words per line) and has

picked up a few words like "he" and "his." However, it has not learned good English grammar, yet it is grasping the statistics of how Shakespeare wrote sentences.

Sample Number: 65 Loss: 1.59179826
METy:
And think, my life yes with full-bripers:
For from she down more trunk one what plain as if.

Report: At this middle stage, the model has a loss of 1.59, and the sample is more structured. Words are condensed, and the network has learned where punctuation should be. “And think, my life yes // as if” are real words, so it is accessing the key in the data. We can still see some words being invented, and grammar still does not make sense. The RNN has grasped local language patterns and the style of Shakespeare but doesn’t understand semantically yet.

Sample Number: 86 Loss: 1.55487725
S
You arriver up as the powerwarks, it comes;
In which have those become with the will of my bussicie----------

Report: At this later stage, the loss is 1.55, which means the model is getting more ‘fluent.’ It’s parsing a subject and a verb, “it comes // in which have those become,” which is a big difference from earlier stages. However, there’s still a lack of meaning in the lines, and the model is trying to imitate Shakespeare by implementing next-character prediction (similar to keyboards). It has learned words but has no semantic understanding.

3. In many cases, this trained network can exhibit striking “emergent” properties of human language. At the end of training, what similarities and differences do you see between the outputs of your model and the competence of a typical adult human? What

if any, changes to the network, to the objective, and/or the training data could close that gap?

At the end of training, the network shows the pattern of how a sentence in English looks like with punctuation and poetic structures that imitate Shakespeare. It can produce a few short words (mainly pronouns) and a few short, grammatically correct sentences—all of which are emerging from how the RNN is learning through statistics and next character prediction.

There are few differences between the RNNs and human language competence. The network consistently invents nonsense words and cannot memorize the structure that comes before the current word, resulting in a lack of meaning. Unlike humans, it doesn’t understand syntax and cannot infer context in a sentence, thereby being unable to produce narrative in long lines.

Transformers that we learned about in week 5 would be helpful in making changes to the network. They use self-attention so that they process input in a parallel way, instead of sequentially like RNNs. They are also better at looking at context that gives a fluent language ability.

4. How do you think this *learning* process (next-character prediction) is similar to or different from a human child learning language?

The way RNNs work with next-character prediction is similar to a human child learning language: (1) children need to learn sequences (where the subject, object, and a noun lie) and (2) use the current word to predict what comes next. They learn from experience like the RNNs being trained on data, and both get better at grasping patterns in language.

However, children do not learn language by just focusing on the prediction of the next character. They have the ability to refer back to the previous word and lines of sentences and think about the meaning of a sentence before speaking it, and they regularly get feedback from teachers. They can also generalize well by applying limited past experience to a broader set of rules, while the RNNs require large datasets and longer training times.

Optional Bonus. Find a different dataset (not Shakespeare) and try again. Some commonly available natural language datasets are Wikipedia Entries and Fairy Tales. If you are fluent in another language, you might find something in that language.

I decided to run an RNN in Uzbek. I was able to find this corpus on the internet as my primary dataset. After briefly reading a few of the lines, I saw a pattern: some of them are common phrases in the Uzbek culture, while some are excerpts from Uzbek literature. It's tricky because Uzbek is a Turkic language, meaning that it operates in a Subject-Object-Verb (SOV) format, and we have the option to drop personal pronouns. As a Turkic language, Uzbek is a null-subject, agglutinative and has no noun classes (gender or otherwise) (Wikipedia, 2026).

Sample Number: 10 Loss: 2.31504973
von Pakolmodir.
Buni oldik.
Oganga bilan o‘lganini bo‘lga bo‘p borga kurra kavodmanni tartim.
Menini ----------

Report: The network has learned a few words and short sentences like "Buni oldik," which means we got it. The way it has learned apostrophes, punctuation, and agglutination early on is impressive.

Sample Number: 68 Loss: 1.62480773
mki, lekin u ko‘rganimyagachiligini janosi yo‘li muvaffaq.
Men gaplashdim.
Katta bordingizmigi ovozda----------

Report: Even though the loss function is higher for Uzbek than for English, it is still succeeding

in producing individual correct words, such as “lekin (but) // yo’li muvaffaq (the path is bright).” It has learned the idea of ‘one sentence is one complete thought’ by demonstrating short sentences usually in past tense, “Men gaplashdim" (I spoke). For me, it feels the model is better at Uzbek data than Shakespeare.

Sample Number: 222 Loss: 1.36109488
ilmang.
Kechasi tomonda.
U vazifasi bu yerdan iboratlashtirmadi.
U odov mast bo‘lganini bildiradi.
To----------

Report: In this later stage, the loss function is 1.36, and sentences seem to follow one structure as one sentence per line. I’m impressed how it’s learned a few phrases correctly, like “mast bo'lmoq" (to be drunk).